INSTITUTO FEDERAL
Paraná
Campus Pinhais

INSTITUTO FEDERAL DO PARANÁ · CAMPUS PINHAIS
Bacharelado em Ciência da Computação

Monitoramento de Funções Virtualizadas de Redes

Um estudo comparativo entre ferramentas de extração de comportamento: eBPF, Sysstat, Prometheus e API do Docker

Autores: **João Pedro dos Santos Henrique Plinta** · **Rafael Correia Alves**
Orientador: Prof. Me. Guilherme Werneck de Oliveira

Defesa de Trabalho de Conclusão de Curso · Pinhais · 2026

Roteiro da apresentação

1. Contexto: NFV e o monitoramento de VNFs
2. Problema, hipótese e objetivos
3. As quatro abordagens comparadas
4. Trabalhos relacionados e lacuna de pesquisa
5. Metodologia e arquitetura experimental
6. Resultados (480 execuções oficiais)
7. Análise de sensibilidade da vazão (+480 execuções)
8. Síntese, conclusões e trabalhos futuros

Virtualização de Funções de Rede (NFV)

- A **NFV desacopla** funções de rede de hardwares proprietários e passam a rodar como software (**VNFs**) em servidores comuns.
- Exemplos de VNFs: firewall, IDS/IPS, balanceador de carga, UPF do 5G e **WAF** (Web Application Firewall).
- A elasticidade depende de **monitoramento contínuo**: CPU, memória, vazão e latência.
- O **MANO** (Management and Orchestration, do ETSI) é a camada que gerencia o ciclo de vida das VNFs: usa essas métricas para decidir escalar, migrar ou recuperar uma VNF em tempo hábil.

O ponto-chave no ambiente NFV

Métrica atrasada → orquestrador reage tarde → **violação de SLA**.

O observador não é externo: roda **co-localizado à VNF**, disputando os mesmos núcleos de CPU e memória do host.

A literatura mostra que o monitoramento pode sobrecarregar a VNF.

INSTITUTO FEDERAL
Paraná
Campus Pinhais

Problema de pesquisa e hipótese

Questão de pesquisa

A escolha da ferramenta impacta o gerenciamento e o **tempo de tomada de decisão em redes vivas**: quais são as diferenças práticas de desempenho entre **eBPF, Sysstat, Prometheus e API do Docker** no monitoramento de uma VNF, e qual oferece **menor sobrecarga**?

Hipótese

O **eBPF**, por operar em nível de **kernel** via kprobes, apresenta menor tempo de resposta e menor sobrecarga em comparação às abordagens em espaço de usuário:

- **Sysstat e Prometheus:** polling via leitura e parsing de /proc;
- **API do Docker:** consulta sob demanda aos cgroups intermediada pelo daemon dockerd.

Objetivos

Objetivo Geral

Compreender como as tecnologias **eBPF, Sysstat, Prometheus e API do Docker** influenciam, do ponto de vista do **tempo de resposta de extração** e da **sobrecarga**, o monitoramento de uma VNF sob diferentes condições de consumo.

Objetivos Específicos

- Analisar os trabalhos da literatura que envolvem monitoramento no contexto de NFV;
- Implementar quatro coletores (eBPF, Sysstat, Prometheus e API do Docker) para monitorar um WAF;
- Executar experimentos quantitativos em quatro volumes de carga (100k, 500k, 1M e 2M), medindo tempo de resposta, CPU e memória;
- Compreender as vantagens, limitações e trade-offs operacionais de cada abordagem.

INSTITUTO FEDERAL
Paraná
Campus Pinhais

As quatro abordagens comparadas

1. eBPF (CO-RE)

Kernel-level

kprobes em tcp_sendmsg (TX) e tcp_cleanup_rbuf (RX), sport=8080.

Acumula bytes em mapa BPF ARRAY no kernel; sem cópia de pacotes.

Compilado sem BCC/LLVM em runtime (~15 MB RSS).

2. Sysstat (/proc)

User-space (polling)

Lê e faz parsing de /proc/net/dev (interface lo) a cada sonda.

Consulta CPU e memória do WAF via psutil (/proc/<pid>).

Abordagem leve, consolidada e sem privilégios de kernel.

3. Prometheus

User-space + HTTP

Mesma coleta do Sysstat (/proc/net/dev + psutil).

Mantém servidor HTTP na porta :8000 expondo /metrics (Gauges).

Avalia o custo de manter o exporter ativo em memória.

4. API do Docker

User-space → Daemon

SDK Python consulta stats(one_shot=True) via /var/run/docker.sock.

O daemon dockerd lê cgroups v2 (CPU/memória) e serializa JSON.

Dispensa pid:host, mas passa pelo IPC do daemon.

Contraste arquitetural: ler mapa em memória do kernel (eBPF) vs. parsing de /proc (Sysstat/Prometheus) vs. IPC HTTP/Unix + cgroups via daemon (Docker).

INSTITUTO FEDERAL
Paraná
Campus Pinhais

Trabalhos relacionados e lacuna

Referência	Foco / Tecnologias Analisadas	Métricas de Sobrecarga	Contexto / VNF
Cassagnes et al. (2020)	eBPF (kprobes) vs. polling em espaço de usuário	Latência de instrumentação	Rede geral / Host
Casalicchio (2017) / Wiedner (2023)	Medição em Docker e latência de cgroups v1/v2	CPU, disco, latência de cauda	Contêineres / NFV
Dinga et al. (2023)	ELK Stack, Netdata, Prometheus e Zipkin	Consumo de energia, CPU, RAM	Microsserviços em Docker
Kannan (2024) / Shahinfar (2025)	Agentes eBPF e isolamento sob concorrência	CPU, memória, vazão, latência	Kubernetes / Host
Oliveira et al. (2026)	Impacto da latência de coleta (Sysstat /proc)	Latência de coleta, reação MANO	NFV / DPI simulado
Este trabalho (2026)	**eBPF + Sysstat + Prometheus + API do Docker**	**RTT UDP, CPU e RAM (WAF e coletor)**	**VNF (WAF isolado), 100k–2M**

Lacuna: nenhum trabalho compara as **quatro abordagens** sobre uma mesma VNF, com carga crescente e rigor estatístico. Este trabalho preenche essa lacuna.

Metodologia e ambiente experimental

- **Classificação:** pesquisa aplicada, quantitativa, comparativa e experimental.
- **VNF avaliada:** WAF em Python (asyncio + ThreadPoolExecutor) que inspeciona **SQLi, XSS, Path Traversal, RCE e Null Byte**.
- Por concentrar carga intensiva de CPU na inspeção de conteúdo, o WAF torna visível qualquer sobrecarga imposta pelo observador.
- **Isolamento:** 4 pilhas Docker Compose independentes; versões de bibliotecas fixadas nos Dockerfiles.

Ambiente Experimental (Host Dedicado)

CPU: AMD Ryzen 5 5500 (6 cores / 12 threads, 4,27 GHz)

RAM: 16 GB DDR4

SO: Ubuntu 26.04 LTS em **modo texto** (multi-user.target)

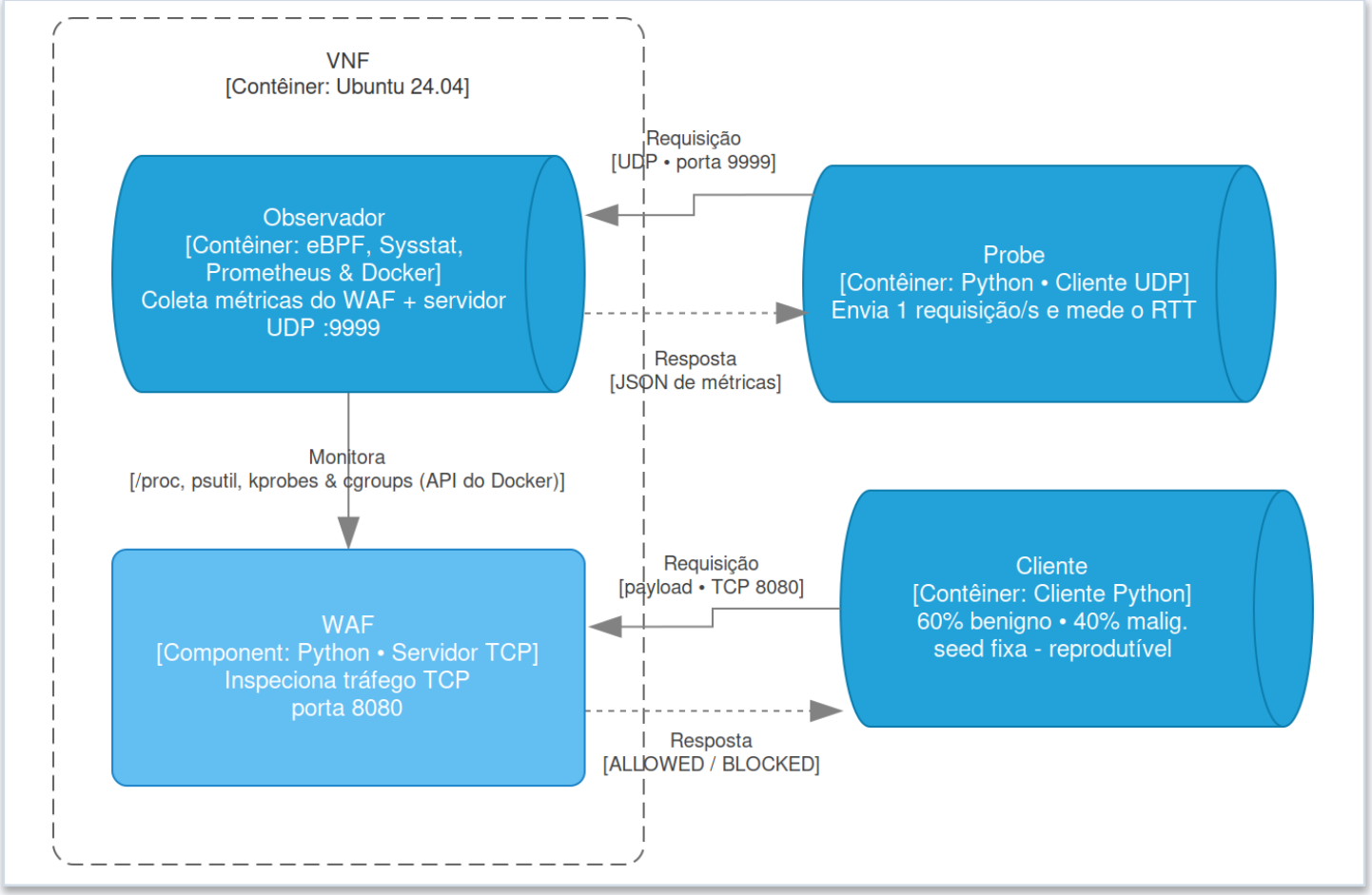
Kernel: Linux 7.0.0-38-generic (com BTF)

Docker Engine: 29.8.2

Stack eBPF: libbpf 1.3.0 + clang 18.1.3 (CO-RE)

Bibliotecas: Python 3.12.3, psutil 7.2.2, prometheus_client 0.25.0, docker SDK 7.2.0

Arquitetura experimental (Diagrama C4)



Fluxo de comunicação e medição

1. Cliente (TCP :8080)

200 corrotinas asyncio com conexões persistentes; framing de 4 bytes big-endian.

2. WAF (VNF)

Inspeciona cada payload (ALLOWED / BLOCKED) e grava waf_metrics.json atomicamente a cada 100 requisições.

3. Observador (UDP :9999)

Coleta métricas com a ferramenta em teste e responde às sondas com JSON.

4. Probe (probe.py, 1 req/s)

Mede o RTT UDP com time.time_ns(), a métrica primária de sobrecarga.

INSTITUTO FEDERAL
Paraná
Campus Pinhais

Protocolo de testes e rigor estatístico

- **Variável independente (N):** 100.000, 500.000, 1.000.000 e 2.000.000 mensagens.
- **Cargas determinísticas:** 60% benignas + 40% maliciosas, pré-geradas com **seed=42** (sequência idêntica entre ferramentas).
- **Janela de coleta:** DURATION = ⌊N / 15000⌋ + 20 s (26, 53, 86 e 153 s): taxa nominal de 15 mil msg/s, mais 20 s de margem para subir e encerrar os contêineres.
- **Repetições:** 30 execuções independentes por cenário → **480 execuções oficiais** (4 ferramentas × 4 volumes × 30).
- **Validação e reprodutibilidade:** escrita atômica em waf_metrics.json (0 falhas nas 480 execuções) + dashboard em **vnf-lab.up.railway.app**.

Métricas e Análise Estatística

Métrica Primária

• Tempo de resposta do observador (RTT UDP, em ms).

Métricas Secundárias (Recursos)

• Memória e CPU do próprio observador;

• CPU e memória do processo WAF (controle).

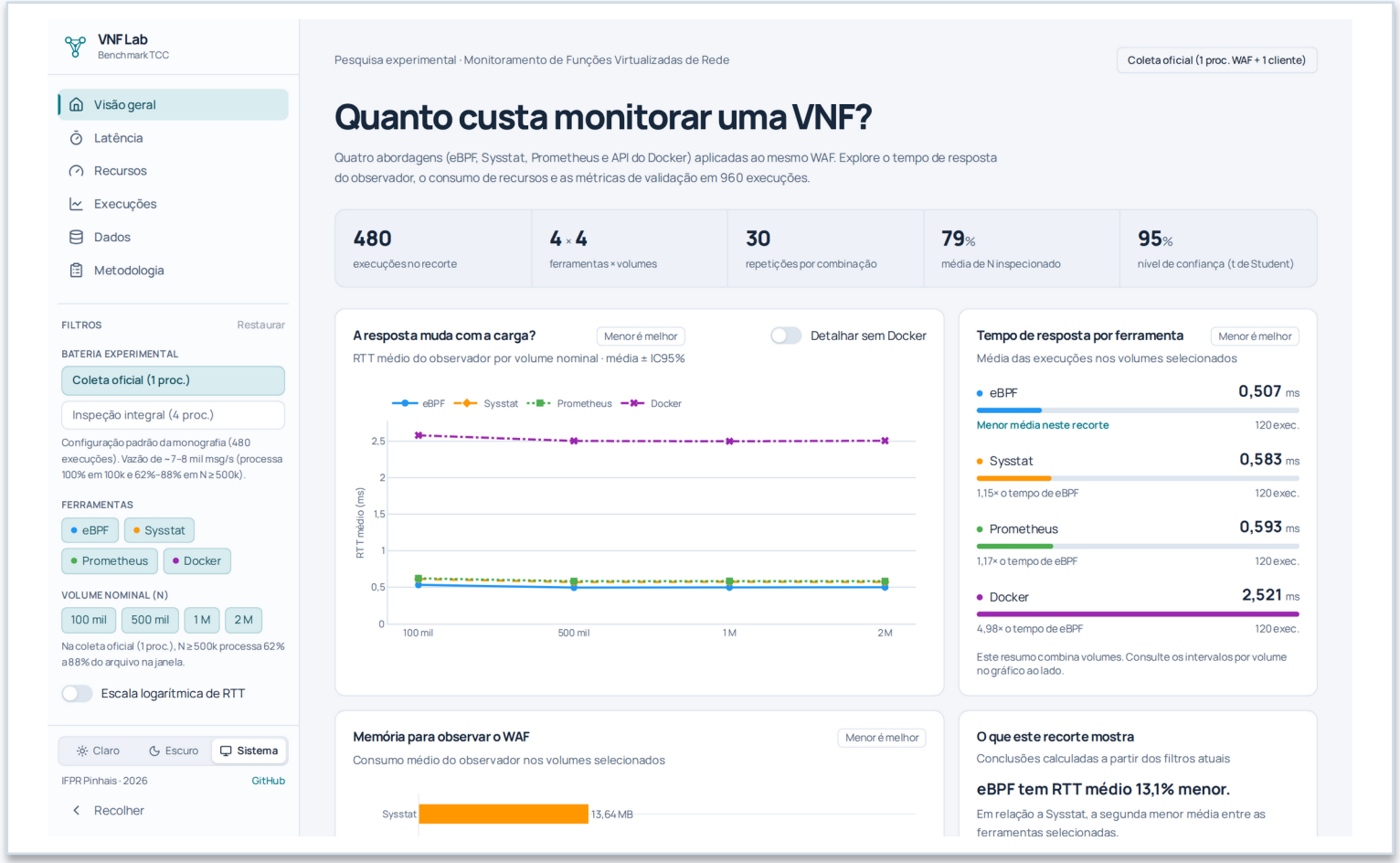
Métricas de Validação

• Tempo médio de inspeção (~0,080 ms) e contadores RX/TX.

Critério Estatístico (t de Student, α = 0,05)

• IC95% = 2,045 × s / √30. Sem sobreposição de IC95% ⇒ diferença significativa.

Dashboard interativo dos resultados



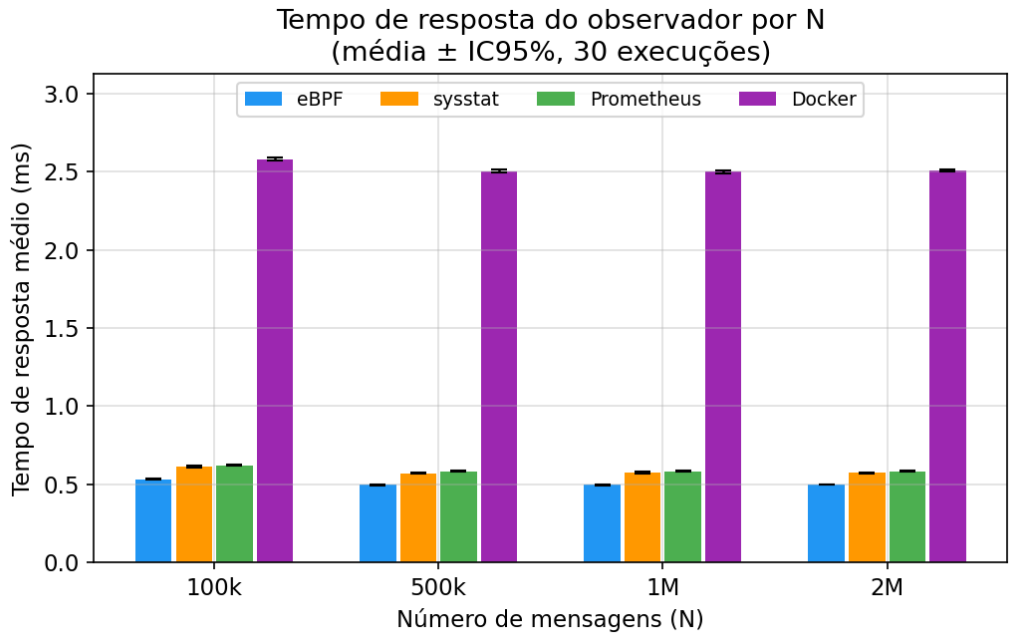
Acesse e explore

vnf-lab.up.railway.app

Acesso público, sem instalar nada.

Aponte a câmera do celular

Tempo de resposta (RTT UDP)



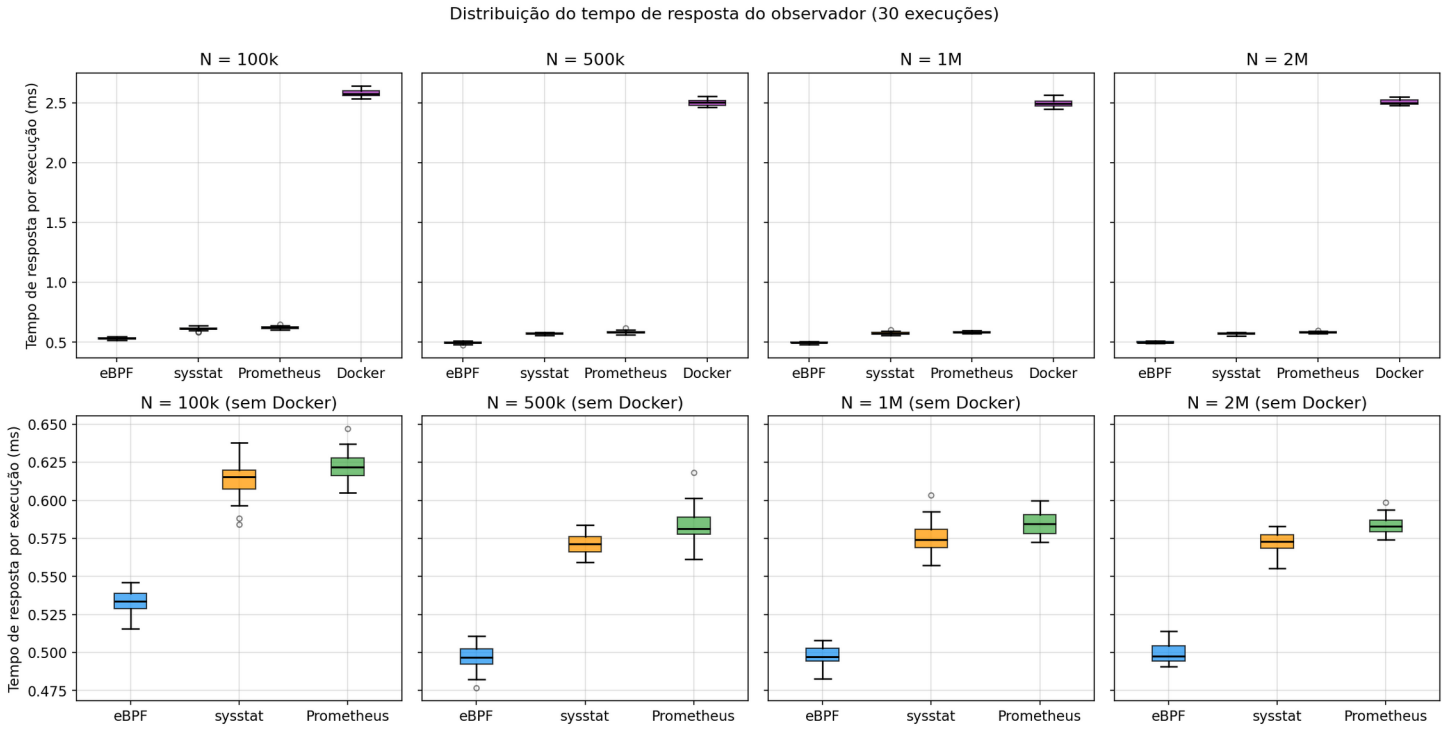
Valores em ms (média ± IC95%, n = 30). Menor é melhor.

Volume (N)	eBPF	Sysstat	Prometheus	Docker
100.000	**0,533 ± 0,003**	0,614 ± 0,005	0,623 ± 0,004	2,579 ± 0,011
500.000	**0,496 ± 0,003**	0,571 ± 0,003	0,583 ± 0,004	2,503 ± 0,010
1.000.000	**0,498 ± 0,002**	0,575 ± 0,004	0,585 ± 0,003	2,498 ± 0,011
2.000.000	**0,500 ± 0,002**	0,573 ± 0,002	0,584 ± 0,002	2,506 ± 0,008

Principais achados (IC95% nunca se sobrepõem)

- **eBPF lidera em todos os volumes:** −13% vs. Sysstat, −15% vs. Prometheus e −80% vs. Docker.
- **Por quê?** Lê contadores prontos em mapa no kernel, sem parsear /proc.
- **Estável:** a partir de 500k o RTT independe do volume.

Distribuição do RTT e custo do Docker



Distribuição (boxplot)

- **eBPF sempre abaixo** de Sysstat e Prometheus.
- Desvio-padrão de apenas **1% a 2% da média** nas 30 execuções.

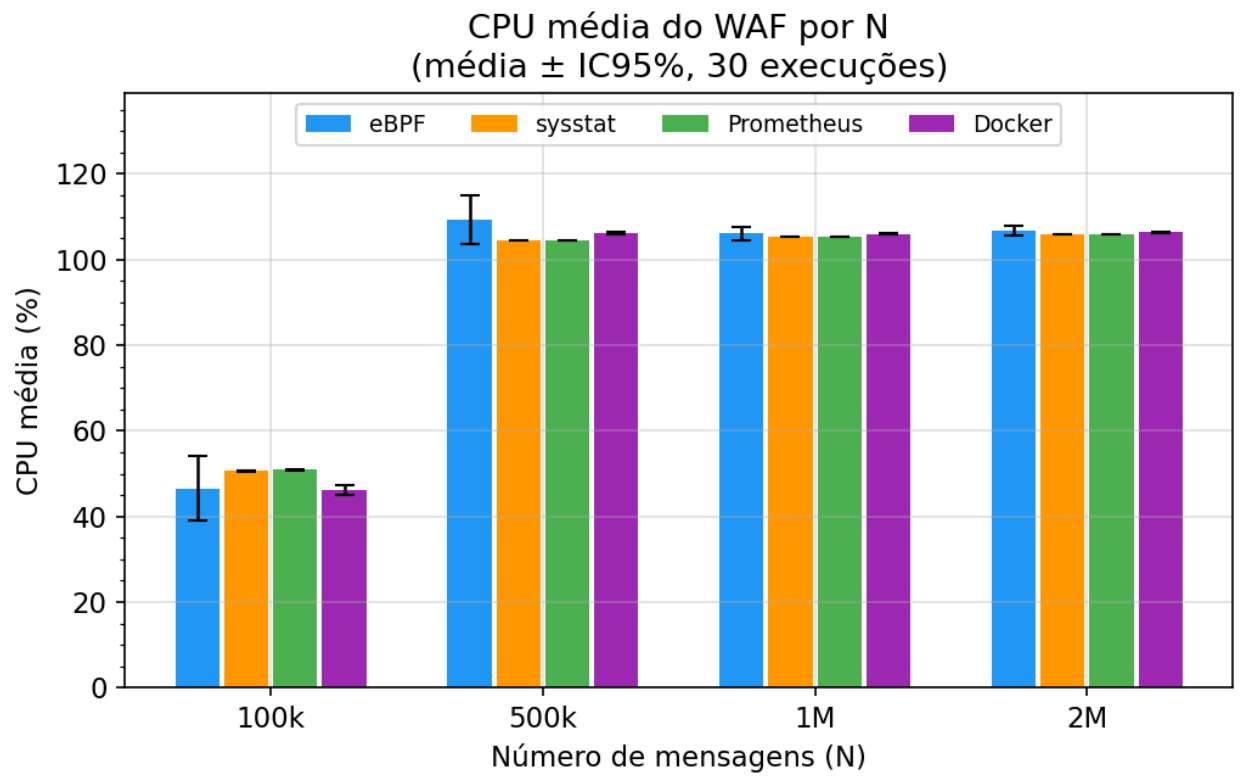
Por que o Docker leva ~2,5 ms?

SDK → socket → daemon → cgroups → JSON, a cada sonda.

Análise isolada das etapas:

- Leitura direta do cgroup: **0,04–0,05 ms**
- Chamada vazia (/_ping): **0,5–0,6 ms**
- stats(one_shot): **1,4–1,7 ms (~30× mais)**

Consumo de CPU do processo WAF



Análise de interferência sobre a VNF

- A partir de N = 500k, o WAF satura em **~104% a 110%** de um núcleo lógico em todos os cenários.
- **Em N = 1M:** 106,1% (eBPF), 105,3% (Sysstat), 105,3% (Prometheus) e 106,1% (Docker).
- Tempo médio de inspeção por payload convergiu para **0,080 ms** nas 4 ferramentas.
- **Conclusão de controle:** nenhum coletor interfere de forma relevante na capacidade de processamento do WAF.
- *Ressalva metodológica: no Docker a CPU vem do cgroup (não do psutil), não sendo estritamente idêntica na fonte.*

INSTITUTO FEDERAL
Paraná
Campus Pinhais

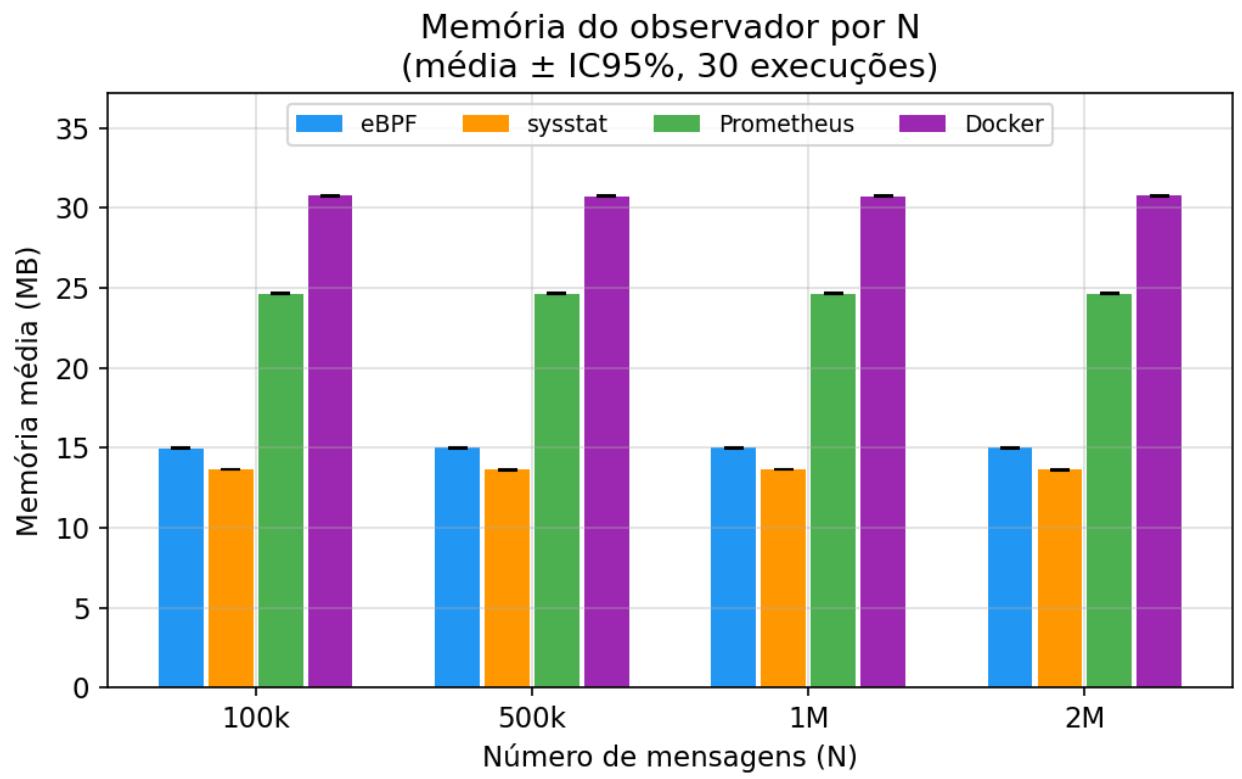
Consumo de CPU do próprio observador

Volume (N)	eBPF (%)	Sysstat (%)	Prometheus (%)	Docker (%)
100.000	2,97 ± 5,97	0,06 ± 0,01	5,05 ± 7,09	0,70 ± 1,19
500.000	0,05 ± 0,00	2,50 ± 3,47	0,06 ± 0,00	0,79 ± 0,95
1.000.000	1,62 ± 2,25	0,83 ± 1,59	2,30 ± 3,39	1,33 ± 0,92
2.000.000	0,56 ± 1,04	0,93 ± 1,24	0,92 ± 1,22	0,21 ± 0,19

Evidência qualitativa: inconclusiva do ponto de vista quantitativo

- Médias de 0,05% a 5,05%, com **IC95% da ordem da própria média** na maioria dos grupos.
- Só 3 grupos ficaram baixos e estáveis (~0,05–0,06%): eBPF e Prometheus em 500k, Sysstat em 100k.
- Causa: amostragem via psutil a cada 1 s e contenção de CPU entre contêineres.
- No Docker, o cgroup é lido pelo **dockerd**, fora do psutil do observador.

Memória do observador e do WAF



Memória do Observador (constante com N)

- **Sysstat: 13,6 MB (menor consumo)**
- **eBPF:** 15,0 MB (−92% vs. ~196 MB do antigo BCC)
- **Prometheus:** 24,7 MB (+81%, servidor HTTP)
- **Docker:** 30,8 MB (+126%, SDK docker)

Memória do WAF

- RSS via psutil: de **~28 MB (100k) a ~46 MB (2M)**, igual entre coletores.
- Docker usa cgroup: grandeza distinta, não comparável.

Sensibilidade da vazão (100% de N)

Gargalo: na coleta oficial o WAF (1 processo) não dá conta da vazão e inspeciona só 62% a 88% de N para N ≥ 500k. Multiplicamos os processos (proc.) e repetimos a bateria, com 30 repetições por ferramenta e volume (+480 execuções):

eBPF, N = 1M	WAF	Cliente	Inspeções	% de N
Original (oficial, 30 rep.)	1 proc.	1 proc.	706.407	71%
Cliente multiprocesso	1 proc.	4 proc.	601.527	60%
WAF e cliente multiproc.	4 proc.	4 proc.	650.067	65%
WAF e cliente 4 proc. + leitura direta	**4 proc.**	**4 proc.**	**1.000.000**	**100%**

Volume (N)	eBPF	Sysstat	Prometheus	Docker
100 mil	0,533 → 0,900	0,614 → 1,018	0,622 → 1,031	2,579 → 2,645
500 mil	0,496 → 0,903	0,571 → 1,012	0,583 → 1,035	2,503 → 2,648
1 milhão	0,498 → 0,900	0,575 → 1,014	0,585 → 1,037	2,498 → 2,658
2 milhões	0,500 → 0,902	0,572 → 1,016	0,584 → 1,037	2,506 → 2,650

Conclusão: 100% inspecionado, ordem preservada

- eBPF (~0,90 ms) < Sysstat (~1,01) < Prometheus (~1,04) < Docker (~2,65), com IC95% estreito.
- O aumento de ~0,4 ms vem do psutil consultar 4 processos waf.py por sonda; o Docker faz 1 chamada.

Síntese comparativa das quatro abordagens

Dimensão	eBPF	Sysstat	Prometheus	API do Docker
Tempo de resposta (RTT)	**Menor (~0,50 ms)**	Intermediário (~0,57 ms)	Intermediário (~0,58 ms)	Maior (~2,50 ms)
CPU do WAF	Equivalente	Equivalente	Equivalente	Equivalente (via cgroup)
Memória do observador	Baixa (~15,0 MB)	**Menor (~13,6 MB)**	Alta (~24,7 MB)	Maior (~30,8 MB)
CPU do observador	Inconclusiva	Inconclusiva (ruidosa)	Inconclusiva (ruidosa)	Inconclusiva (externa)
Requisitos de privilégio	Privilegiado + BTF + pid:host	Não priv. + pid:host	Não priv. + pid:host	Socket docker.sock (sem pid:host)

eBPF

Menor RTT (−13% a −80%) e pouca memória. Exige privilégio de kernel.

Sysstat

Menor memória (~13,6 MB), simples e só ~0,07 ms atrás do eBPF.

Prometheus

Paga ~11 MB de RAM pelo servidor HTTP; vale se integrado ao scraping.

API do Docker

Isola por contêiner, mas custa 5× mais RTT e ~31 MB de RAM.

Conclusões

- **Hipótese confirmada na métrica primária:** por operar no kernel, o eBPF obteve o menor tempo de resposta em todos os volumes (−13% vs. Sysstat, −15% vs. Prometheus e −80% vs. Docker), sem sobrecarregar o WAF.
- **Ponderação prática (trade-off):** o ganho sobre o polling de /proc é estatisticamente significativo, porém moderado em termos absolutos (0,07 a 0,09 ms), devendo ser pesado frente à maior complexidade do eBPF (C, compilação, privilégios).
- **O custo das camadas de abstração:** a API do Docker mostrou que o gargalo (~2,5 ms) não está no kernel (cgroup), mas no IPC via socket e serialização pelo daemon dockerd.
- **Contribuições do trabalho:** benchmark aberto e reprodutível com 960 execuções (480 oficiais + 480 de sensibilidade), migração BCC → libbpf CO-RE (−92% de RAM) e dashboard interativo público.

INSTITUTO FEDERAL
Paraná
Campus Pinhais

Trabalhos futuros

- **Leitura direta de cgroups v2:** coletar de /sys/fs/cgroup/ sem passar pelo dockerd, para isolar o custo da API do Docker (~2,5 ms).
- **Ambiente de produção:** repetir o experimento em rede física e com o volume de pacotes de produção, não só em loopback.
- **Impacto fim a fim na VNF:** medir como o monitoramento afeta a latência e a vazão das requisições dos clientes do WAF.
- **WAF de alta vazão em processo único:** atingir 100% de N sem multiplicar os processos monitorados pelo psutil.
- **Novos cenários:** cadeias SFC completas, Prometheus com scraping ativo e hooks XDP/TC no eBPF.

INSTITUTO FEDERAL
Paraná
Campus Pinhais

Obrigado!

Dúvidas e considerações da banca

João Pedro dos Santos Henrique Plinta
Rafael Correia Alves

Orientador: Prof. Me. Guilherme Werneck de Oliveira

Código-fonte e dados
github.com/joaopedroplinta/vnf-monitoring-benchmark

Dashboard interativo
vnf-lab.up.railway.app